
Security Review Report
NM-0945 - Blockful ENS Security Council



NETHERMIND
SECURITY

(June 5, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	Veto Flow	4
4.2	Extension Flow	4
4.3	Renounce Flow	4
5	Risk Rating Methodology	5
6	Documentation Evaluation	6
7	Test Suite Evaluation	7
7.1	Tests Output	7
8	About Nethermind	8

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for Blockful ENS Security Council contract. The SecurityCouncil contract is a single-purpose access control component that grants a multisig the time-bounded authority to cancel pending proposals queued in the ENS governance timelock. The contract holds the PROPOSER_ROLE on the OpenZeppelin TimelockController used by the ENS DAO and exposes a single veto entry point gated by an expiration timestamp, which can be extended only through a DAO governance vote and is permissionlessly renounced once it elapses.

The audit comprises 40 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, (b) automated analysis tools, and (c) creation of test cases. **Along this document, we report zero points of attention, as shown in Fig. 1.**

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 discusses the documentation provided by the client for this audit. Section 7 presents the compilation, tests, and automated tests. Section 8 concludes the document.

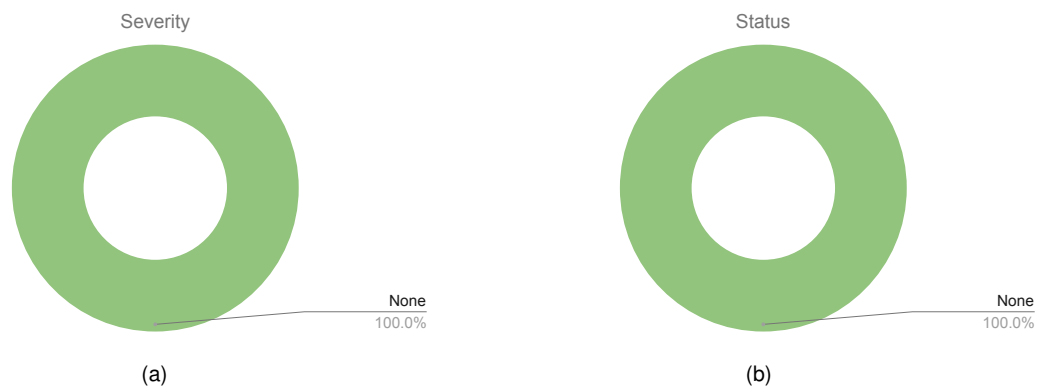


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (0), Undetermined (0), Informational (0), Best Practices (0). Distribution of status: Fixed (0), Acknowledged (0), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	June 4, 2026
Final Report	June 4, 2026
Initial Commit	e196ad7
Final Commit	e196ad7
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	SecurityCouncil.sol	40	29	72.5%	10	79
	Total	40	29	72.5%	19	79

3 Summary of Issues

No issues were identified during the review.

4 System Overview

The SecurityCouncil contract gives a multisig the time-bounded authority to cancel proposals queued in the ENS DAO's OpenZeppelin TimelockController. It is granted PROPOSER_ROLE on the timelock by a governance vote after deployment and uses that role to call cancel on the multisig's behalf. An expiration timestamp bounds the council's lifetime. The DAO can move it forward through a governance vote, and once it has elapsed, anyone can renounce the role permissionlessly.

The contract inherits from ReverseClaimer, used to register an ENS reverse record at construction, and Ownable2Step, which provides owner management. The constructor calls _transferOwnership directly so the multisig is the owner immediately at deployment.

4.1 Veto Flow

The multisig can cancel a queued proposal through the veto function:

```
function veto(bytes32 proposalId) external onlyOwner;
```

The function requires `block.timestamp < expiration` and forwards the call to `timelock.cancel(proposalId)`. Cancellation is permitted on the timelock because the SecurityCouncil contract holds the PROPOSER_ROLE, which on OpenZeppelin's TimelockController is also the role authorized to cancel pending operations.

4.2 Extension Flow

The DAO can extend the council's term via the extend function:

```
function extend(uint256 newExpiration) external;
```

The caller must be the timelock, and `newExpiration` must be strictly greater than both the current expiration and `block.timestamp`. The contract must still hold PROPOSER_ROLE on the timelock. If the role has been renounced, the DAO must re-grant it before extending.

4.3 Renounce Flow

Once `block.timestamp >= expiration`, any caller can invoke `renounceTimelockRoleByExpiration`, which calls `timelock.renounceRole(PROPOSER_ROLE, address(this))`.

```
function renounceTimelockRoleByExpiration() external;
```

Making this step permissionless ensures the council can be wound down without relying on the multisig signers to cooperate near expiry.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about ENS Security Council documentation

The documentation for this audit consisted primarily of inline NatSpec comments within the `SecurityCouncil` contract, alongside a `README` file that provided a high-level overview of the contract, its core features, and the operational lifecycle covering deployment, role granting, vetoing, term extension, and post-expiration role renunciation.

7 Test Suite Evaluation

7.1 Tests Output

```
> forge test

Ran 1 test for test/integration/security-council/constructor.t.sol:Constructor_Integration_Concrete_Test
[PASS] test_MultisigIsOwner_AtDeployment() (gas: 13329)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 884.25ms (2.70ms CPU time)

Ran 2 tests for test/integration/security-council/renounceTimelockRoleByExpiration.t.sol:
RenounceTimelockRoleByExpiration_Integration_Concrete_Test
[PASS] test_RenounceTimelockRoleByExpiration_Success() (gas: 26681)
[PASS] test_RenounceTimelockRoleByExpiration_TooEarly() (gas: 24211)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.05s (236.96ms CPU time)

Ran 10 tests for test/integration/security-council/extend.t.sol:Extend_Integration_Concrete_Test
[PASS] test_Extend_AfterOriginalExpiration_BeforeRenounce() (gas: 31346)
[PASS] test_Extend_ByTimelock_Success() (gas: 23993)
[PASS] test_RenounceTimelockRole_AtNewExpiration() (gas: 29601)
[PASS] test_Revert_Extend_AfterRoleRenounced() (gas: 29698)
[PASS] test_Revert_Extend_ByRandomUser() (gas: 14832)
[PASS] test_Revert_Extend_BySecurityCouncilMultisig() (gas: 14853)
[PASS] test_Revert_Extend_WithEqualExpiration() (gas: 14787)
[PASS] test_Revert_Extend_WithLowerExpiration() (gas: 14927)
[PASS] test_Revert_Extend_WithPastTimestamp() (gas: 17729)
[PASS] test_Revert_RenounceTimelockRole_BeforeNewExpiration() (gas: 24278)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 1.05s (243.89ms CPU time)

Ran 2 tests for
↳ test/fork/security-council/renounceTimelockRoleByExpiration.t.sol:RenounceTimelockRoleByExpiration_Fork_Test
[PASS] test_RenounceTimelockRoleByExpiration_Success() (gas: 23970)
[PASS] test_RenounceTimelockRoleByExpiration_TooEarly() (gas: 21877)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.07s (384.41ms CPU time)

Ran 1 test for test/AttackDAO.t.sol:Attack_DAO_Test
[PASS] test_Attack_DAO() (gas: 579212)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.72s (835.46ms CPU time)

Ran 4 tests for test/integration/security-council/veto.t.sol:Veto_Integration_Concrete_Test
[PASS] test_Revert_When_Queue_After_Veto() (gas: 251707)
[PASS] test_Revert_When_Security_Council_Vetoing_After_Expiration() (gas: 261186)
[PASS] test_Security_Council_Vetoing() (gas: 184498)
[PASS] test_Veto_Random_User_Trying_Veto() (gas: 25031)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.72s (135.93ms CPU time)

Ran 5 tests for test/integration/security-council/extend.t.sol:Extend_With_Queue_Proposal_Integration_Concrete_Test
[PASS] test_Extend_VetoCancelsRealProposal_PastOriginalExpiration() (gas: 38977)
[PASS] test_Revert_When_Queue_After_Veto() (gas: 251707)
[PASS] test_Revert_When_Security_Council_Vetoing_After_Expiration() (gas: 261186)
[PASS] test_Security_Council_Vetoing() (gas: 184498)
[PASS] test_Veto_Random_User_Trying_Veto() (gas: 25031)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.72s (136.50ms CPU time)

Ran 4 tests for test/fork/security-council/veto.t.sol:Veto_Fork_Test
[PASS] test_Revert_When_Queue_After_Veto() (gas: 249216)
[PASS] test_Revert_When_Security_Council_Vetoing_After_Expiration() (gas: 258577)
[PASS] test_Security_Council_Vetoing() (gas: 182007)
[PASS] test_Veto_Random_User_Trying_Veto() (gas: 24583)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.79s (373.91ms CPU time)

Ran 8 test suites in 1.80s (11.00s CPU time): 29 tests passed, 0 failed, 0 skipped (29 total tests)
```

8 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.